TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN A DISTANCIA

UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL

METODOLOGÍA DE SISTEMAS II

Trabajo Práctico Integrador

"Academa"

Sistema de Gestión de Solicitudes Académicas

Evolución profesional de un sistema existente:

calidad, deuda técnica, refactoring y revisión técnica

Estudiante: Andrelo, Luciano Aldo (Comisión: N°7)

Estudiante: Bohorquez Tello, Tomas Esteban (Comisión: N°6)

Estudiante: Carro, Matias Manuel (Comisión: N°11)

Estudiante: Maldonado, Ariana (Comisión: N°10)

Fecha de entrega: 31/10/2026

Índice

1. Resumen ejecutivo....1
2. Descripción del sistema base....1
2.1 Nombre y dominio....1
2.2 Actores y entidades....1
2.3 Operaciones y reglas....2
2.4 Tecnología, pantalla y persistencia....4
2.5 Las dos versiones y su ejecución....4
3. Calidad externa y calidad interna....8
3.1 Calidad externa....8
3.2 Calidad interna....8
3.3 Impacto en el mantenimiento y evolución del sistema....9
4. Deuda técnica y code smells....9
5. SOLID y Clean Code....10
5.1 Principios aplicados....10
Principio abierto/cerrado (OCP)....11
5.2 Principios revisados sin necesidad de cambios específicos....11
5.3 Aplicación de Clean Code....12
6. Refactorings....13
6.1 Extract Method....13
6.2 Move Method....13
6.3 Introduce Parameter Object....14
6.4 Replace Conditional with Strategy y State....15
7. Pruebas y verificación....16
7.1 Pruebas sobre la versión inicial....17
7.2 Casos - Comparación con versión refactorizada....19
7.3 Resultado....20
7.4 Cómo repetir las pruebas....22
8. Code review....22
8.1 Observaciones....24
9. Conclusiones....25
9.1 Conclusión grupal....25
9.2 Conclusiones individuales....25
10. Repositorio, tags y video....26
11. Uso de inteligencia artificial....27
12. Anexo de ejecución....28

1. Resumen ejecutivo

Academa es un sistema de gestión de solicitudes académicas de alcance acotado. Permite registrar solicitudes, clasificarlas por tipo, prioridad y estado, asignar responsables, realizar búsquedas y generar reportes.

El trabajo compara dos versiones del sistema: una inicial, que concentra gran parte de la lógica en la clase Gestor, y otra refactorizada, que distribuye las responsabilidades entre distintas clases. El objetivo es mejorar la organización, legibilidad y mantenibilidad del código, conservando el comportamiento esperado del sistema.

La comparación automática evaluó 17 operaciones y la generación del reporte. En las pruebas realizadas, ambas versiones produjeron los mismos resultados, lo que aporta evidencia de que las refactorizaciones preservaron el comportamiento evaluado.

2. Descripción del sistema base

2.1 Nombre y dominio

El sistema se denomina Academa y tiene como objetivo gestionar solicitudes académicas. Permite registrar trámites de tipo Constancia, Equivalencia y Revision, asignarles una prioridad y un estado, designar responsables y consultar la información mediante búsquedas y reportes.

El alcance del sistema se limita a estas operaciones, dado que el propósito del trabajo es analizar y mejorar la calidad interna del código mediante técnicas de refactorización, conservando el comportamiento de las funcionalidades existentes.

Las dos versiones mantienen la misma interfaz y permiten realizar las mismas operaciones. Para distinguirlas durante la ejecución, se identifican como versión inicial y versión refactorizada, tanto en la consola como en la interfaz, según corresponda.

2.2 Actores y entidades

El sistema contempla dos actores principales: el estudiante, que inicia la solicitud y se identifica mediante su nombre y legajo, y el responsable, que se encarga de gestionar el trámite y puede representar a un área o a un área junto con una persona asignada.

La solicitud es el elemento central del sistema. Contiene la información necesaria para identificar y gestionar cada trámite: número identificador, tipo, prioridad, descripción y estado.

La siguiente tabla compara cómo se representan los principales conceptos del dominio en la versión inicial y en la versión refactorizada.

Concepto	Versión inicial	Versión refactorizada
Estudiante	Datos de nombre y legajo almacenados como columnas de texto.	Clase Estudiante.
Solicitud	Datos del trámite almacenados en una línea del archivo.	Clase Solicitud.
Responsable	Nombre almacenado como texto o un guion cuando no hay responsable asignado.	Clase Responsable.
Estado	Texto con los valores: - Nueva, En curso, Resuelta o Rechazada.	EstadoSolicitud y sus subclases.
Tipo	Texto con los valores: - Constancia, Equivalencia o Revisión.	TipoSolicitud y sus implementaciones específicas.

Estas entidades y estructuras representan los conceptos centrales del sistema y permiten cumplir el requisito de contar con al menos tres entidades o estructuras principales. En la versión refactorizada, su separación favorece una mejor distribución de responsabilidades y facilita la comprensión y el mantenimiento del código.

2.3 Operaciones y reglas

Academa contempla cinco operaciones principales, cuyo comportamiento debe mantenerse en ambas versiones. Los mensajes de respuesta se reproducen exactamente como los devuelve el código, ya que forman parte de los resultados que se comparan durante las pruebas.

1. Registro de solicitudes

Para registrar una solicitud, se deben completar los siguientes campos:

- **Datos obligatorios:** nombre, legajo, tipo, prioridad y descripción. Si falta alguno, el sistema responde "Faltan datos de la solicitud".
- **Tipos admitidos:** Constancia, Equivalencia y Revision. Cualquier otro valor genera el mensaje "Tipo de solicitud no valido". Aunque la interfaz muestra "Revisión", el valor enviado al sistema es Revision, sin tilde.
- **Prioridades válidas:** Alta, Media y Baja. Si se ingresa otra opción, el sistema responde "Prioridad no valida".
- **Resultado del registro:** la solicitud queda en estado Nueva, sin responsable asignado —representado mediante un guion—, y el sistema responde "Solicitud registrada".

2. Cambio de estado

Las transiciones permitidas siguen una secuencia definida:

- Nueva → En curso.

- En curso → Resuelta o Rechazada.
- **Estados finales:** Resuelta y Rechazada no admiten nuevas transiciones.

Si el cambio es válido, el sistema responde "Estado actualizado". Si la transición no está permitida, devuelve "Esa transicion de estado no esta permitida".

3. Asignación de responsables

- **Validación:** el responsable no puede quedar vacío. En ese caso, el sistema responde "El responsable no puede estar vacio" y no modifica el archivo de datos.
- **Asignación correcta:** si la solicitud existe y la asignación es válida, responde "Responsable asignado".
- **Estado de la solicitud:** la asignación no modifica su estado.
- **Solicitud inexistente:** si el identificador no existe, el sistema responde "No se encontro la solicitud".

4. Búsqueda de solicitudes

- **Filtros disponibles:** tipo, prioridad y estado.
- **Filtros vacíos:** no restringen los resultados.
- **Respuesta:** Gestor devuelve la lista correspondiente y deja vacío el mensaje de respuesta.

La interfaz informa la cantidad de solicitudes encontradas.

5. Generación de reportes

El reporte presenta un renglón por solicitud e incluye los siguientes datos: identificador, nombre, tipo, prioridad, estado, responsable, etiqueta y marca.

- **Etiquetas por tipo:**
  - **Constancia:** TRAMITE.
  - **Equivalencia**: ACADEMICO.
  - **Revision:** EVALUACION.
- **Marcas por prioridad:**
  - **Alta:** URGENTE.
  - **Media:** NORMAL.
  - **Baja:** BAJA.

Estos valores forman parte del comportamiento que ambas versiones deben conservar para garantizar la equivalencia de los resultados evaluados.

2.4 Tecnología, pantalla y persistencia

El sistema está desarrollado en **Java** y cuenta con una interfaz web construida con **HTML, CSS y JavaScript**. Los archivos de la interfaz se encuentran en la carpeta web:

- **index.html:** estructura de la página.
- **estilos.css:** presentación visual.
- **app.js:** comportamiento de la interfaz.

La clase **interfaz.Servidor** publica la página en la propia computadora, mediante la dirección 127.0.0.1, y abre el navegador. El sistema no emplea una base de datos ni un framework de persistencia; los datos se almacenan en archivos de texto codificados en UTF-8.

Formato de almacenamiento

Cada solicitud ocupa una línea del archivo y sus campos se separan mediante el carácter **|.**

El orden de los datos es el siguiente:

id | nombre | legajo | tipo | prioridad | descripción | estado | responsable | etiqueta | marca

Este formato impone una restricción importante: **la descripción no debe contener el carácter |**, ya que podría dividir incorrectamente la línea y desplazar el contenido de los campos posteriores.

Los nombres empleados en las pruebas, como Ana Perez, Luis Gomez y Mora Diaz, son ficticios y se utilizan únicamente con fines de verificación.

Diferencias entre las versiones

- **Versión inicial:** escucha en el **puerto 8081** y almacena las solicitudes en **solicitudes-inicial.txt.**
- **Versión refactorizada:** escucha en el **puerto 8082** y utiliza el archivo **solicitudes-refactorizada.txt.**

La interfaz incluye tres entradas de navegación: Inicio, Solicitudes y Reportes. Las operaciones se realizan desde la misma página. La interfaz no determina si una transición de estado es válida, sino que delega esa decisión en Gestor y presenta la respuesta recibida.

2.5 Las dos versiones y su ejecución

El proyecto contiene dos implementaciones del sistema, organizadas en directorios separados:

- **src/inicial:** contiene la versión de partida. La **clase Gestor** concentra las operaciones de procesamiento, cambio de estado, asignación de responsables, búsqueda y generación de reportes.

- **src/refactorizado:** contiene la versión mejorada, que conserva las operaciones disponibles y reorganiza las responsabilidades entre **distintas clases.**

- **src/pruebas/PruebasInicial.java:** ejecuta seis pruebas funcionales automatizadas sobre la versión inicial.

- **src/pruebas/Comparacion.java:** ejecuta las mismas entradas en ambas implementaciones y compara sus resultados.

Para ejecutar y verificar el proyecto, se incluyen los siguientes archivos por lotes (.bat):

- **ejecutar-inicial.bat:** inicia la versión inicial.
- **ejecutar-refactorizada.bat:** inicia la versión refactorizada.
- **compilar-y-probar.bat:** compila el código y ejecuta la comparación automática entre ambas versiones.

Cada versión mantiene abierta una ventana de consola mientras el servidor está en funcionamiento. Esta ventana se cierra al finalizar la ejecución del programa.

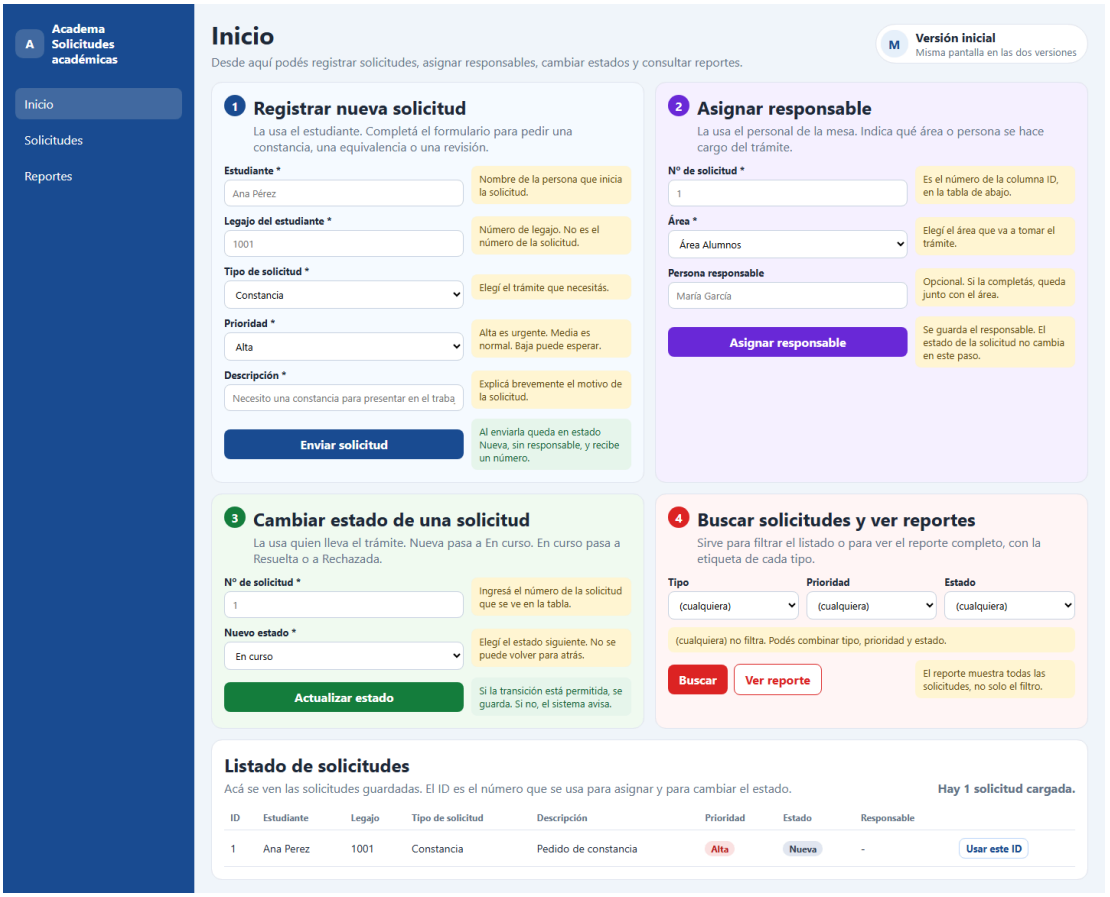


Figura 1. *Versión inicial de Academa después de registrar una solicitud de prueba.*

La pantalla muestra la solicitud con ID 1, correspondiente a Ana Perez (legajo 1001), de tipo Constancia, prioridad Alta y estado Nueva, con la descripción "Pedido de constancia" y sin responsable asignado. El menú lateral identifica el sistema como Academa y el indicador superior señala que se está ejecutando la versión inicial. La versión refactorizada conserva la misma interfaz y modifica dicho indicador para identificar la implementación en ejecución.

Diagrama de la Versión Inicial

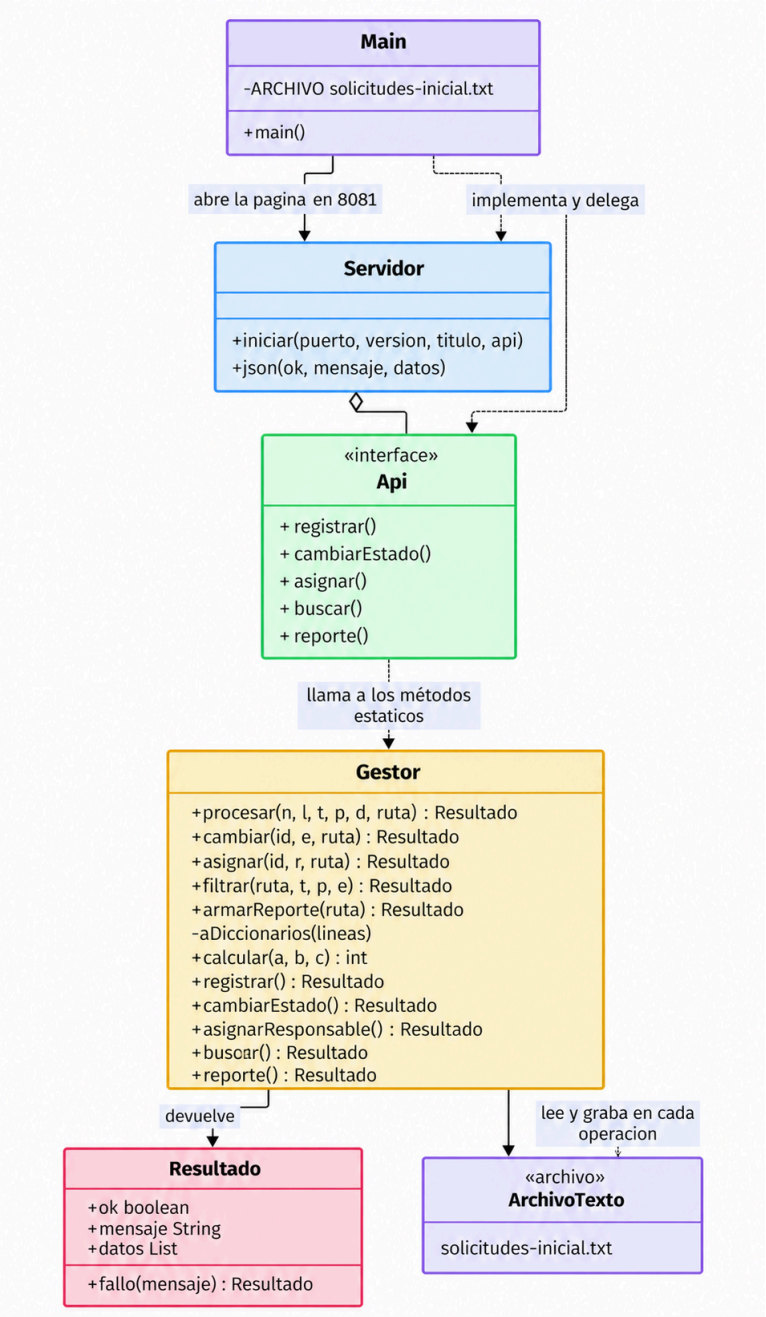


Figura 2. *Diagrama de clases de la versión inicial de Academa.*

Main inicia el servidor web mediante Servidor en el puerto 8081. La **interfaz Api** delega las operaciones en **Gestor**, que concentra la lógica de registro, cambio de estado, asignación de responsables, búsqueda y generación de reportes, además del acceso al archivo de solicitudes. El método calcular permanece en Gestor, aunque no se detectaron llamadas a él. El elemento **ArchivoTexto** representa el archivo de persistencia solicitudes-inicial.txt, no una clase Java del programa.

Diagrama de clases de la Versión Refactorizada

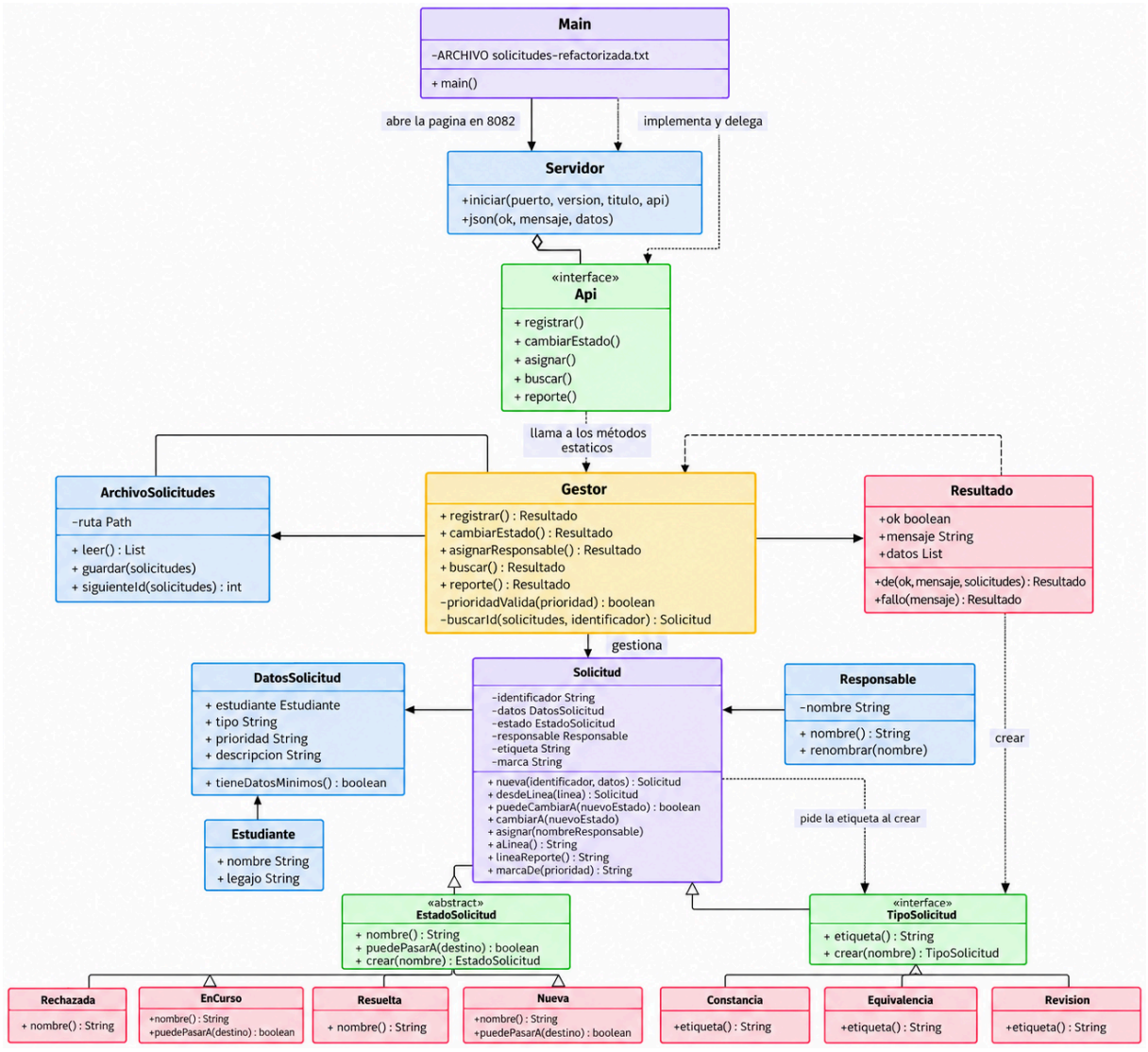


Figura 3. *Diagrama de clases de la versión refactorizada de Academa.*

Main inicia el servidor web mediante **Servidor** en el puerto 8082. La interfaz Api delega las operaciones en **Gestor,** que centraliza la coordinación del sistema. La persistencia se delega en **ArchivoSolicitudes**; los datos del registro se agrupan en **DatosSolicitud,** y las responsabilidades se

distribuyen entre **Solicitud** y las clases relacionadas. Las implementaciones de **TipoSolicitud** determinan las etiquetas de cada tipo de trámite, mientras que las subclases de **EstadoSolicitud** definen las transiciones permitidas. Los estados **Resuelta** y **Rechazada** heredan el comportamiento que impide nuevas transiciones. Esta organización reduce la concentración de lógica en Gestor y facilita el mantenimiento del código.

3. Calidad externa y calidad interna

3.1 Calidad externa

La calidad externa se refiere al comportamiento observable del sistema desde la perspectiva del usuario, sin necesidad de examinar su código fuente.

En Academa, se puede comprobar mediante las siguientes operaciones:

- **Registro:** al cargar una solicitud válida, el sistema asigna un identificador y la registra con estado Nueva y sin responsable asignado.
- **Cambio de estado:** permite pasar de Nueva a En curso y, posteriormente, a Resuelta o Rechazada. Las transiciones no permitidas son rechazadas por el sistema.
- **Búsqueda y asignación:** permite consultar solicitudes mediante filtros y asignar responsables, respetando las validaciones correspondientes.
- **Generación de reportes:** presenta los datos de las solicitudes registradas.

Las dos versiones mantienen la misma interfaz y deben producir los mismos resultados ante las mismas entradas. Por lo tanto, la refactorización no busca incorporar funcionalidades nuevas ni modificar la experiencia del usuario, sino mejorar la estructura interna del programa sin alterar su comportamiento esperado.

3.2 Calidad interna

La calidad interna se relaciona con la organización del código, la distribución de responsabilidades, la legibilidad y la facilidad de mantenimiento.

En la versión inicial, la **clase Gestor** concentra gran parte de la lógica del sistema. El método procesar (líneas 16 a 93) combina validaciones, generación de identificadores, asignación de etiquetas y marcas, escritura en el archivo y preparación de información para el reporte. Además, contiene lógica cuyo resultado no se utiliza posteriormente.

Otros métodos, como cambiar, asignar, filtrar y **armarReporte**, vuelven a acceder al archivo, lo que evidencia una distribución poco centralizada de las responsabilidades. También se identifican bloques catch vacíos y un método calcular que no presenta llamadas detectadas.

Esta concentración dificulta la comprensión del código y aumenta el esfuerzo necesario para modificarlo sin afectar otras operaciones.

En la versión refactorizada, las responsabilidades se distribuyen entre clases específicas. Por ejemplo, las transiciones de estado se delegan en **EstadoSolicitud,** mediante el método **puedePasarA.** De esta manera, las reglas se expresan en el componente correspondiente, lo que facilita su lectura y mantenimiento.

3.3 Impacto en el mantenimiento y evolución del sistema

La diferencia entre ambas versiones se hace más evidente al considerar posibles modificaciones futuras.

En la **versión inicial**, las transiciones de estado se resuelven mediante condicionales dentro del método cambiar. Si se necesitara incorporar un nuevo estado, sería necesario revisar esa lógica y verificar que los cambios no afectaran el resto del procesamiento concentrado en Gestor.

En la **versión refactorizada**, las transiciones se gestionan mediante clases específicas que representan los estados. Por ejemplo, para incorporar un estado hipotético denominado Observada, sería necesario agregar su representación correspondiente y definir las transiciones permitidas, además de actualizar el método crear para que pueda instanciarlo. Las operaciones de registro y persistencia podrían mantenerse sin cambios si la nueva funcionalidad no modifica sus responsabilidades.

El estado Observada no forma parte de las funcionalidades actuales de Academa. Se utiliza únicamente como ejemplo para mostrar cómo la distribución de responsabilidades puede reducir el alcance de las modificaciones futuras.

4. Deuda técnica y code smells

La deuda técnica de Academa se concentra principalmente en la clase Gestor de la versión inicial, ubicada en src/inicial/Gestor.java. En ella se identifican nueve *code smells* o indicadores de problemas de diseño y mantenimiento.

La siguiente tabla detalla cada caso, su ubicación en el código, las dificultades que puede generar ante futuras modificaciones y la prioridad sugerida para su corrección. Las referencias a líneas corresponden a la versión inicial analizada.

Smell	Ubicación	Evidencia	Impacto y riesgo	Prioridad
Long Method	procesar, líneas 16 a 93	Valida, lee el archivo, calcula el id, elige etiqueta y marca, graba y arma un texto que no devuelve.	Cualquier regla obliga a releer todo el método. Es fácil tocar una regla vecina.	Alta
Long Parameter List	Línea 16	procesar recibe n, l, t, p, d y la ruta por separado.	Es fácil invertir nombre y legajo, o tipo y prioridad.	Alta

Mysterious Name	Líneas 16 y 17, y el resto de la clase	n, l, t, p, d, aux, x, s, e y r no dicen qué representan.	Hay que leer el cuerpo para entender la variable.	Media
God Class	Clase Gestor	Valida, persiste, cambia el estado, asigna, busca y arma el reporte.	Un cambio reabre toda la clase y arrastra al resto.	Alta
Duplicated Code	Lecturas en 30, 98, 160, 195 y 225	La lectura está copiada en cinco métodos. La grabación se repite en las líneas 69, 147 y 186.	Un cambio de formato hay que repetirlo en varios lugares.	Alta
Condicional largo	Líneas 43 a 60 y 112 a 124	Hay un if por tipo, otro por prioridad y otro por transición de estado.	Un tipo o un estado nuevo se edita en el método central.	Alta
Excepción escondida	Catch vacíos en 31, 70, 99, 148, 161, 187, 196 y 226	Esos catch no informan el fallo. El de la línea 90 no está vacío: devuelve «Faltan datos de la solicitud».	Un archivo dañado parece una lista vacía, o un error parece un dato faltante.	Alta
Dead Code	calcular, líneas 271 a 279	Ninguna operación lo llama. Suma importes según el texto ALTA o MEDIA.	Parece que el sistema calcula importes. El riesgo de hoy es bajo.	Baja
Regla en el lugar incorrecto	armarReporte, líneas 234 a 243	Calcula una etiqueta mirando partes[2], que es el legajo. El renglón usa partes[8] y partes[9].	El cálculo usa una columna incorrecta y no modifica el reporte. Esta lógica redundante dificulta el mantenimiento.	Media

El noveno caso requiere analizar la estructura del archivo de datos y la lógica de **armarReporte**.

Las columnas se organizan de la siguiente manera: la posición 0 corresponde al identificador, la 1 al nombre, la 2 al legajo y la 3 al tipo de solicitud.

Sin embargo, armarReporte evalúa la columna 2 para determinar si el tipo es Constancia, Equivalencia o Revision. Como esa posición contiene el legajo, la condición no se cumple con los datos de prueba y la variable local conserva la etiqueta previamente almacenada durante el registro. El renglón del reporte utiliza esa etiqueta y la marca correspondiente.

Por este motivo, la comparación entre ambas versiones no detecta diferencias en el reporte. El problema es interno: existe una regla redundante que evalúa una columna incorrecta y cuyo resultado no modifica la información finalmente presentada.

5. SOLID y Clean Code

5.1 Principios aplicados

Principio de responsabilidad única (SRP)

La comparación entre ambas versiones de Gestor permite observar la aplicación del principio de responsabilidad única. En la versión inicial, una misma clase concentra tareas relacionadas con las **validaciones, la persistencia, los cambios de estado, los tipos de solicitud y la generación de reportes**. Esto hace que una modificación en cualquiera de estas áreas pueda requerir cambios en la misma clase.

En la versión refactorizada, las responsabilidades se distribuyen entre clases específicas: **ArchivoSolicitudes** se encarga de leer y guardar los datos, **EstadoSolicitud** define las transiciones permitidas, **TipoSolicitud** determina la etiqueta correspondiente y Solicitud reúne los datos y comportamientos propios del trámite. De esta manera, Gestor coordina las operaciones y delega las tareas en las clases responsables de cada función, **reduciendo los motivos de cambio de cada componente**.

Principio abierto/cerrado (OCP)

La aplicación de este principio se observa en la gestión de los tipos y estados de las solicitudes. Para incorporar un nuevo tipo, se agrega una clase que implementa **TipoSolicitud** y se incorpora su selección en el método **TipoSolicitud.crear**. Así, el método de registro deja de contener condicionales específicos para determinar cada etiqueta.

De manera similar, un nuevo estado requiere una subclase de **EstadoSolicitud** y su incorporación en **EstadoSolicitud.crear**. Por lo tanto, la solución mejora la separación de responsabilidades, aunque la selección de las clases continúa dependiendo de métodos de creación que deben modificarse al incorporar nuevas opciones. El avance consiste en **evitar que la lógica principal de registro y gestión de estados tenga que conocer los detalles de cada tipo o estado**.

5.2 Principios revisados sin necesidad de cambios específicos

Principio de sustitución de Liskov (LSP)

La revisión de este principio no identificó un defecto que requiriera una modificación específica. Los estados Nueva, EnCurso, Resuelta y Rechazada se utilizan como implementaciones de **EstadoSolicitud,** y el método **puedePasarA** determina si una transición está permitida. Los estados Resuelta y Rechazada heredan del método base la respuesta que impide nuevas transiciones. De este modo, **el código que consulta el estado puede trabajar con la abstracción sin depender de una clase concreta**.

Principio de segregación de interfaces (ISP)

Tampoco se identificó la necesidad de modificar las interfaces. **TipoSolicitud** declara el método etiqueta, mientras que **EstadoSolicitud** define nombre y **puedePasarA.** Cada abstracción reúne

operaciones relacionadas con su responsabilidad, **sin obligar a las implementaciones a definir métodos que no necesitan**.

Principio de inversión de dependencias (DIP)

La solución mantiene una dependencia directa con el mecanismo de persistencia elegido: el archivo de texto. Gestor crea y utiliza **ArchivoSolicitudes** sin recurrir a una interfaz de repositorio. Dado que el alcance actual contempla un único mecanismo de almacenamiento, se decidió no incorporar una abstracción adicional que no respondiera a una necesidad concreta del sistema. Esta decisión mantiene una estructura sencilla, aunque implica que **sustituir el mecanismo de persistencia podría requerir cambios en las clases que dependen de él**.

5.3 Aplicación de Clean Code

La refactorización mejora la **legibilidad** mediante nombres que expresan con mayor claridad la intención del código. Identificadores genéricos como n, l, t, p, d, aux y x se reemplazan por nombres más descriptivos, como DatosSolicitud, tieneDatosMinimos, puedePasarA, etiqueta y ArchivoSolicitudes.

También se busca que cada método tenga una **responsabilidad concreta**. Por ejemplo, **tieneDatosMinimos** permite comprobar si están presentes los cinco datos requeridos para el registro, mientras que **puedePasarA** determina si una transición de estado es válida. Esta organización facilita la comprensión de las reglas sin necesidad de recorrer métodos extensos.

Manejo de errores

El manejo de errores también presenta una mejora. Si el archivo no existe, **ArchivoSolicitudes.leer** devuelve una lista vacía, interpretando que todavía no hay solicitudes almacenadas. Si se produce otro error durante la lectura o la escritura, se lanza una **IllegalStateException** con un mensaje descriptivo: "No se pudo leer el archivo de solicitudes" o "No se pudo guardar el archivo de solicitudes". Esto evita que los errores de entrada y salida queden ocultos, como ocurría con los bloques catch vacíos de la versión inicial.

Por último, las **reglas de negocio quedan ubicadas en las clases correspondientes**: el tipo determina su etiqueta, el estado define las transiciones permitidas y Gestor solicita las operaciones de persistencia a **ArchivoSolicitudes.** Esta distribución reduce la concentración de lógica en una única clase y evita repetir el acceso al archivo en cada operación.

6. Refactorings

Se documentan cuatro refactorings: Extract Method, Move Method, Introduce Parameter Object y Replace Conditional with Strategy y State. Las pruebas del apartado 7 permiten verificar que los cambios preservan el comportamiento evaluado del sistema.

6.1 Extract Method

Antes de la refactorización, la validación de los cinco datos obligatorios estaba incluida directamente en el método procesar (línea 25). La condición comprobaba que nombre, legajo, tipo, prioridad y descripción no estuvieran vacíos. Si faltaba alguno, el sistema devolvía el mensaje "Faltan datos de la solicitud".

- `if (!n.equals("") && !l.equals("") && !t.equals("") && !p.equals("") && !d.equals("")) {`

Esta validación estaba mezclada con otras tareas del registro, como el manejo del archivo y la generación del identificador.

Después de la refactorización, la comprobación se trasladó al método tieneDatosMinimos, de la clase DatosSolicitud. De este modo, Gestor consulta el resultado de la validación y determina la respuesta cuando faltan datos.

```
if (!datos.tieneDatosMinimos()) {
   return Resultado.fallo("Faltan datos de la solicitud");
}
```

Se aplicó la técnica **Extract Method** para separar la validación de las demás tareas del registro. Así, la condición queda encapsulada en un método con un nombre descriptivo y se conserva el mensaje de error original.

6.2 Move Method

Antes de la refactorización, el método cambiar, ubicado entre las líneas 112 y 124, determinaba las transiciones de estado mediante condicionales anidados. Por su parte, el método procesar asignaba la etiqueta correspondiente al tipo de solicitud entre las líneas 43 y 51. Ambas reglas estaban concentradas en Gestor, junto con las operaciones de acceso al archivo.

El siguiente fragmento muestra cómo se evaluaban las transiciones a En curso y Resuelta. La transición a Rechazada seguía una lógica similar: solo se permitía cuando el estado actual era En curso.

```
if (e.equals("En curso")) {
   if (actual.equals("Nueva")) {
      permitido = true;
```

```
    }
} else if (e.equals("Resuelta")) {
    if (actual.equals("En curso")) {
        permitido = true;
    }
}
```

Después de la refactorización, cada clase de estado define las transiciones que admite. El estado Nueva permite pasar a En curso:

```
public boolean puedePasarA(String destino) {
    return "En curso".equals(destino);
}
```

El estado EnCurso permite pasar a Resuelta o Rechazada:

```
public boolean puedePasarA(String destino) {
    return "Resuelta".equals(destino) || "Rechazada".equals(destino);
}
```

Los estados Resuelta y Rechazada heredan la implementación de EstadoSolicitud, que impide nuevas transiciones. A su vez, Solicitud delega la comprobación en su estado actual:

```
public boolean puedeCambiarA(String nuevoEstado) {
    return estado.puedePasarA(nuevoEstado);
}
```

La misma técnica se aplicó a otras responsabilidades: la determinación de la etiqueta pasó a las implementaciones de TipoSolicitud (Constancia, Equivalencia y Revision), mientras que las operaciones de lectura y escritura se trasladaron a ArchivoSolicitudes.

Se aplicó **Move Method** para ubicar cada comportamiento en la clase que corresponde a su responsabilidad. De esta manera, Gestor deja de concentrar estas reglas y el código resulta más organizado y fácil de mantener.

6.3 Introduce Parameter Object

Antes de la refactorización, el método procesar recibía seis argumentos independientes, identificados con nombres poco descriptivos. Cinco correspondían a los datos de la solicitud y el sexto indicaba la ruta del archivo donde se almacenaban.

- public Resultado procesar(String n, String l, String t, String p, String d, String ruta)

Esta forma de pasar los datos dificultaba la lectura de la firma del método y aumentaba el riesgo de intercambiar argumentos por error.

Después de la refactorización, los cinco datos de la solicitud se agruparon en el objeto DatosSolicitud. El nombre y el legajo se representan mediante la clase Estudiante, mientras que el tipo, la prioridad y la descripción se almacenan como atributos de DatosSolicitud.

```
public DatosSolicitud(String nombre, String legajo, String tipo,
                      String prioridad, String descripcion) {
   this.estudiante = new Estudiante(limpiar(nombre), limpiar(legajo));
   this.tipo = limpiar(tipo);
   this.prioridad = limpiar(prioridad);
   this.descripcion = limpiar(descripcion);
}
```

El responsable no se incluye en DatosSolicitud, ya que se asigna después del registro y puede modificarse durante la gestión de la solicitud. La ruta del archivo también permanece fuera del objeto, porque corresponde a la persistencia y no a los datos propios del trámite.

Se aplicó **Introduce Parameter Object** para agrupar los datos relacionados en una única estructura con significado dentro del dominio. Esto mejora la legibilidad y reduce el riesgo de errores al pasar argumentos.

6.4 Replace Conditional with Strategy y State

El cuarto refactoring reemplazó las cadenas de condicionales que determinaban el comportamiento según el tipo y el estado de una solicitud. Para ello, se aplicaron los patrones **Strategy** y **State**, asignando una clase a cada variante de comportamiento.

Strategy se aplicó a los tipos de solicitud. Cada implementación de TipoSolicitud define su etiqueta mediante el mismo método:

```
class Constancia implements TipoSolicitud {
   public String etiqueta() {
      return "TRAMITE";
   }
}
```

State se aplicó a los estados de la solicitud. Cada estado define las transiciones permitidas, mientras que los estados terminales impiden nuevas transiciones. Por ejemplo, el estado Nueva solo permite pasar a En curso:

```
class Nueva extends EstadoSolicitud {
   public String nombre() {
      return "Nueva";
   }

   public boolean puedePasarA(String destino) {
      return "En curso".equals(destino);
   }
}
```

Se eligieron estos patrones porque el código presentaba comportamientos que variaban según un valor: la etiqueta correspondiente al tipo de solicitud y las transiciones admitidas por el estado actual. En cambio, la marca de prioridad se mantuvo en el método breve marcaDe, dentro de Solicitud. Como se trata de tres textos fijos, no se consideró necesario crear otra familia de clases.
Tampoco se incorporaron otros patrones, ya que las necesidades de Academa no justificaban su complejidad: el sistema no requiere notificar cambios a múltiples pantallas, encolar órdenes ni adaptar una biblioteca externa.

7. Pruebas y verificación

La verificación del comportamiento de Academa se organizó en dos etapas. En primer lugar, se implementaron seis pruebas funcionales automatizadas sobre la versión inicial, estableciendo una referencia de los resultados esperados para las principales operaciones del sistema.

Posteriormente, se desarrolló una batería comparativa de 17 operaciones sobre ambas implementaciones, incorporando validaciones, cambios de estado, asignación de responsables y búsquedas. Además, se verificó la equivalencia de los reportes generados y el comportamiento ante archivos inexistentes.

Las pruebas se ejecutaron mediante el script compilar-y-probar.bat, permitiendo comprobar que las versiones inicial y refactorizada conservaron los mismos resultados en los escenarios evaluados.

7.1 Pruebas sobre la versión inicial

Como preparación de la etapa de verificación, se implementó una batería inicial de pruebas funcionales automatizadas sobre la versión inicial de Academa, sin modificar el código de *src/inicial/*. Estas pruebas fueron desarrolladas en la clase *src/pruebas/PruebasInicial.java* y se ejecutan mediante el script *compilar-y-probar.bat*.

El objetivo de esta instancia fue establecer una base de comportamiento esperado antes de incorporar la versión refactorizada, de manera que posteriormente sea posible comparar ambas implementaciones y detectar regresiones.

Las pruebas utilizan archivos temporales independientes mediante *Files.createTempFile()*, evitando alterar los datos originales del sistema. Cada caso realiza verificaciones automáticas mediante aserciones y elimina sus archivos de prueba al finalizar.

ID	Caso de prueba	Objetivo	Resultado
P01	Registro de solicitud válida	Verificar que una solicitud correcta se registre y quede en estado Nueva	Superada
P02	Rechazo de solicitud incompleta	Verificar que una solicitud sin nombre sea rechazada y no quede guardada	Superada
P03	Cambio de estado Nueva ⟶ En curso	Verificar una transición de estado permitida y su persistencia	Superada
P04	Rechazo de transición inválida	Verificar que no se permita pasar directamente de Nueva a Resuelta	Superada
P05	Asignación de responsable	Verificar la asignación de un responsable y su persistencia	Superada
P06	Generación de reporte	Verificar que el sistema genere un reporte con la información esperada	Superada

La ejecución completa de estas pruebas finalizó correctamente, proporcionando una referencia funcional sobre la versión inicial del sistema.

Figura 4. Resultado de la ejecución automatizada de las seis pruebas funcionales sobre la versión inicial de Academa

```
====================================
ACADEMA - COMPILACION Y PRUEBAS
====================================

[1/2] Compilando clases...

[2/2] Ejecutando pruebas...

=== INICIANDO PRUEBAS DE ACADEMA ===

Resultado del registro:
¿Registro exitoso? true
Mensaje: Solicitud registrada
Datos: [{id=1, nombre=Estudiante Prueba, legajo=12345, tipo=Constancia, prioridad=Alta,
ble=-}]
PRUEBA P01 SUPERADA: registro válido
Archivo temporal P01 eliminado.

PRUEBA P02 SUPERADA: solicitud incompleta rechazada
Archivo temporal P02 eliminado.

PRUEBA P03 SUPERADA: cambio de estado Nueva a En curso
Archivo temporal P03 eliminado.

PRUEBA P04 SUPERADA: transicion invalida rechazada
Archivo temporal P04 eliminado.

PRUEBA P05 SUPERADA: responsable asignado correctamente
Archivo temporal P05 eliminado.

PRUEBA P06 SUPERADA: reporte generado correctamente
Archivo temporal P06 eliminado.

=== TODAS LAS PRUEBAS SUPERADAS ===

====================================
PRUEBAS FINALIZADAS CORRECTAMENTE
====================================
```

7.2 Casos - Comparación con versión refactorizada

Como continuación de las pruebas funcionales realizadas sobre la versión inicial, se implementó una comparación automatizada entre ambas versiones de Academa mediante la clase src/pruebas/Comparacion.java.

Esta clase ejecuta una secuencia de 17 operaciones equivalentes sobre las implementaciones inicial y refactorizada, utilizando archivos temporales independientes para evitar interferencias entre los datos de ambas versiones.

En cada operación se verifica el resultado esperado y se comparan los valores devueltos por ambas implementaciones, incluyendo el indicador de éxito o rechazo, el mensaje de respuesta y los datos de las solicitudes: identificador, nombre, legajo, tipo, prioridad, descripción, estado y responsable.

La batería comparativa amplió la cobertura funcional respecto de las seis pruebas iniciales, incorporando escenarios de validación, transiciones permitidas y rechazadas, asignación de responsables y búsquedas filtradas.

Paso	Datos	Mensaje
Registrar	Ana Perez, 1001, Constancia, Alta, Pedido de constancia	Solicitud registrada
Registrar	Nombre vacío, legajo 1002, Equivalencia, Media	Faltan datos de la solicitud
Registrar	Luis Gomez, 1002, tipo Otro, Baja	Tipo de solicitud no valido
Registrar	Luis Gomez, 1002, Equivalencia, prioridad Urgente	Prioridad no valida
Registrar	Luis Gomez, 1002, Equivalencia, Media, Pedido de equivalencia	Solicitud registrada
Registrar	Mora Diaz, 1003, Revision, Baja, Revision de parcial	Solicitud registrada
Cambiar 1	En curso	Estado actualizado
Cambiar 1	Nueva	Esa transición de estado no está permitida
Cambiar 2	Resuelta, y la solicitud sigue Nueva	Esa transicion de estado no está permitida
Cambiar 1	Resuelta	Estado actualizado
Cambiar 1	En curso, y ya está Resuelta	Esa transicion de estado no está permitida
Cambiar 99	En curso	No se encontro la solicitud
Asignar 2	Area Alumnos	Responsable asignado
Asignar 2	Responsable vacío	El responsable no puede estar vacío
Asignar 50	Mesa de entrada	No se encontró la solicitud
Buscar	Tipo Equivalencia	Mensaje vacío. La lista contiene la solicitud 2.
Buscar	Constancia y prioridad Baja	Mensaje vacío. La lista queda vacía.

Al finalizar la secuencia, se generan los reportes de los archivos correspondientes a ambas versiones y se verifica también el comportamiento ante la consulta de un archivo inexistente. En este último caso, se comprueba que tanto el mensaje como la lista de resultados estén vacíos. Estas dos comprobaciones se ejecutan por separado y no forman parte de las 17 operaciones comparadas.

7.3 Resultado

La ejecución automatizada se realizó mediante compilar-y-probar.bat, utilizando el entorno Java configurado para el proyecto. El proceso compiló ambas implementaciones y ejecutó las seis pruebas funcionales iniciales, seguidas de las 17 operaciones comparativas.

La consola confirmó también la equivalencia de los reportes generados y del comportamiento ante la consulta de archivos inexistentes, finalizando con el siguiente resultado:

OK: las dos versiones coinciden en 17 operaciones, en la generación de reportes de solicitudes y en el manejo de archivos inexistentes.

TODAS LAS PRUEBAS FINALIZARON CORRECTAMENTE

El reporte final, equivalente en ambas versiones, contiene las solicitudes registradas durante la secuencia de verificación y se presenta a continuación.

ID	Nombre	Tipo	Prioridad	Estado	Responsable	Etiqueta	Marca
1	Ana Perez	Constancia	Alta	Resuelta	-	TRAMITE	URGENTE
2	Luis Gomez	Equivalencia	Media	Nueva	Area Alumnos	ACADEMICO	NORMAL
3	Mora Diaz	Revision	Baja	Nueva	-	EVALUACION	BAJA

Figura 5: Ejecución completa de las pruebas funcionales iniciales y de las 17 operaciones comparativas mediante compilar-y-probar.bat.

```
                                                .\compilar-y-probar.bat
====================================
ACADEMA - COMPILACION Y PRUEBAS
====================================

[1/3] Compilando clases de ambas versiones y pruebas...

[2/3] Ejecutando pruebas de la version inicial...

=== INICIANDO PRUEBAS DE ACADEMA ===

Resultado del registro:
¿Registro exitoso? true
Mensaje: Solicitud registrada
Datos: [{id=1, nombre=Estudiante Prueba, legajo=12345, tipo=Constancia, prioridad=Alta, descrip
ble=-}]
PRUEBA P01 SUPERADA: registro válido
Archivo temporal P01 eliminado.

PRUEBA P02 SUPERADA: solicitud incompleta rechazada
Archivo temporal P02 eliminado.

PRUEBA P03 SUPERADA: cambio de estado Nueva a En curso
Archivo temporal P03 eliminado.

PRUEBA P04 SUPERADA: transicion invalida rechazada
Archivo temporal P04 eliminado.

PRUEBA P05 SUPERADA: responsable asignado correctamente
Archivo temporal P05 eliminado.

PRUEBA P06 SUPERADA: reporte generado correctamente
Archivo temporal P06 eliminado.

=== TODAS LAS PRUEBAS SUPERADAS ===

[3/3] Ejecutando comparacion entre versiones...

=== COMPARACION INICIAL / REFACTORIZADA ===
OK 01 Registrar Constancia
OK 02 Registrar Equivalencia
OK 03 Registrar Otro
OK 04 Registrar Equivalencia
OK 05 Registrar Equivalencia
OK 06 Registrar Revision
OK 07 Cambiar 1 -> En curso
OK 08 Cambiar 1 -> Nueva
OK 09 Cambiar 2 -> Resuelta
OK 10 Cambiar 1 -> Resuelta
OK 11 Cambiar 1 -> En curso
OK 12 Cambiar 99 -> En curso
OK 13 Asignar 2 -> Area Alumnos
OK 14 Asignar 2 ->
OK 15 Asignar 50 -> Mesa de entrada
OK 16 Buscar Equivalencia//
OK 17 Buscar Constancia/Baja/
OK Reporte
OK Archivo inexistente
OK: las dos versiones coinciden en 17 operaciones, en la generación de reportes de solicitudes
y en el manejo de archivos inexistentes.

====================================
TODAS LAS PRUEBAS FINALIZARON CORRECTAMENTE
====================================
```

Los resultados permiten verificar que **Ana** termina en estado Resuelta después de pasar por Nueva y En curso. **Luis** permanece en Nueva y tiene asignado un responsable. **Mora** también permanece en Nueva, con prioridad Baja.

Además, se comprueba que el intento de pasar a Luis directamente a Resuelta no modifica su estado, al igual que el intento de devolver a Ana a En curso. Las **etiquetas y marcas** generadas durante el registro se conservan en las filas del reporte.

Las pruebas comparan los resultados devueltos por Gestor. Por separado, se verifica la generación del reporte de los archivos de ambas versiones y la consulta de un archivo inexistente, para el cual tanto el mensaje como la lista de resultados quedan vacíos.

En la interfaz, el botón **"Ver reporte"** muestra los renglones en un bloque independiente y el aviso cambia a Reporte generado. **La Figura 1** presenta la versión inicial con una sola solicitud cargada. Esta captura sirve como evidencia visual para la defensa, pero no reemplaza la secuencia de pruebas ni los resultados de la comparación.

7.4 Cómo repetir las pruebas

Desde la carpeta raíz del proyecto, se ejecuta el archivo `compilar-y-probar.bat`. Este compila las clases de ambas implementaciones y las clases de prueba utilizando `javac` con codificación UTF-8.

Posteriormente, ejecuta `pruebas.PruebasInicial`, que verifica seis escenarios funcionales sobre la versión inicial, y `pruebas.Comparacion`, que compara 17 operaciones entre ambas implementaciones, además de los reportes y el comportamiento ante archivos inexistentes.

Si alguna prueba falla o se detecta una diferencia, la ejecución finaliza con un error. Si todas las verificaciones son satisfactorias, el script informa que las pruebas finalizaron correctamente.

8. Code review

La revisión de código se realizó sobre las **versiones inicial y refactorizada**, ubicadas en src/inicial y src/refactorizado, respectivamente. Se evaluaron la organización del código, la distribución de responsabilidades, la legibilidad, el manejo de errores y el cumplimiento de los criterios de calidad definidos para el proyecto.

La revisión quedó documentada en los siguientes archivos:

- **docs/checklist-code-review.md:** contiene la lista de verificación aplicada.
- **docs/diagnostico.md:** detalla los problemas identificados.
- **docs/refactorizaciones.md:** registra los cambios realizados y las mejoras introducidas.

Criterio	Versión inicial	Versión refactorizada
Nombres	Los nombres n, l, t, p, d, aux y x dificultan comprender el propósito de las variables sin leer el método completo.	Los nombres representan conceptos concretos, como Estudiante, EstadoSolicitud, etiqueta y ArchivoSolicitudes.
Complejidad y responsabilidades	El método procesar concentra validaciones, generación de identificadores, manejo del archivo, asignación de etiquetas y construcción de respuestas.	Las responsabilidades se distribuyen en métodos específicos, como tieneDatosMinimos, puedePasarA y etiqueta.
Duplicación	La lectura del archivo se repite en cinco métodos y la grabación, en tres.	La lectura y la escritura se centralizan en ArchivoSolicitudes.
Manejo de errores	Contiene varios bloques catch vacíos que pueden ocultar errores de acceso a archivos. Además, un bloque de manejo de errores puede confundir un fallo de lectura con datos faltantes, dificultando la identificación de la causa.	Si el archivo no existe, se devuelve una lista vacía; otros errores se informan mediante una excepción.
Distribución de responsabilidades	La clase Gestor concentra gran parte de la lógica del sistema.	Las responsabilidades se distribuyen entre Solicitud, Estudiante, Responsable, EstadoSolicitud y TipoSolicitud, entre otras clases.
Separación entre interfaz y lógica	La página delega las operaciones en Gestor y muestra los mensajes recibidos.	Se conserva la delegación de las operaciones en Java, sin trasladar las reglas de negocio a la interfaz.
Generación del reporte	armarReporte evalúa una columna incorrecta al intentar determinar la etiqueta a partir del legajo, y el resultado de ese cálculo no se imprime.	lineaReporte utiliza la etiqueta y la marca almacenadas en la solicitud.
Conservación del comportamiento	La versión inicial constituye la referencia para comparar los resultados de las operaciones.	La clase Comparacion verifica que ambas versiones coincidan en las 17 operaciones evaluadas y en el reporte.
Patrones de diseño	La lógica de los tipos y las transiciones de estado no está separada mediante las abstracciones incorporadas en la refactorización.	Se incorporan los patrones **Strategy** para los tipos de solicitud y **State** para los estados, distribuyendo la lógica específica entre sus implementaciones.

El criterio de aceptación de la versión refactorizada se basa en aspectos verificables: los nombres permiten comprender el propósito de cada elemento, las responsabilidades están distribuidas entre clases específicas, las operaciones de lectura y escritura se centralizan en ArchivoSolicitudes, los errores se informan adecuadamente y Comparacion finaliza sin detectar diferencias entre ambas versiones.

La versión inicial presentaba problemas respecto de estos criterios, lo que justificó la refactorización. Por último, el método calcular permanece en la versión inicial como **código muerto**, ya que no se detectaron llamadas a él, y no se encuentra en la versión refactorizada.

8.1 Observaciones

Durante el code review se identificaron tres observaciones técnicas, que fueron evaluadas según su prioridad, impacto y relación con el alcance del proyecto. A continuación, se presentan los hallazgos y las decisiones adoptadas.

ID	Observación	Decisión
CR-01	Posible omisión silenciosa de registros mal formados en ArchivoSolicitudes	Limitación aceptada
CR-02	Constructor de Solicitud con una lista extensa de parámetros	Conservar el diseño actual
CR-03	Necesidad de modificar los métodos crear() para incorporar nuevos tipos o estados	Aceptar la extensibilidad parcial

CR-01: Se detectó que algunos registros mal formados pueden omitirse sin informar el error. Se decidió mantener el comportamiento actual y documentarlo como una posible mejora futura.

CR-02: El constructor de Solicitud recibe varios parámetros, pero el sistema cuenta con un método que facilita su creación. Por eso, no se consideró necesario modificarlo.

CR-03: Para agregar nuevos tipos o estados es necesario modificar los métodos crear(). Se aceptó esta limitación porque Academa utiliza un conjunto reducido de tipos y estados.

Las tres observaciones se consideraron no bloqueantes. Se decidió conservar la implementación actual, dejando documentadas sus limitaciones.

Además, las seis pruebas iniciales y las 17 operaciones comparativas finalizaron correctamente, sin detectar diferencias entre ambas versiones en los casos evaluados.

Resultado del code review: aprobado con observaciones documentadas y aceptadas.

9. Conclusiones

9.1 Conclusión grupal

Academa permite observar la diferencia entre mantener el comportamiento de un sistema y mejorar su calidad interna. Ambas versiones resuelven las mismas operaciones, pero presentan diferencias en su organización y facilidad de mantenimiento. La versión inicial funciona, aunque concentra deuda técnica en un método extenso, nombres poco descriptivos, operaciones de acceso al archivo duplicadas, condicionales asociados a los tipos y estados, y un manejo de errores que puede ocultar fallos.

La versión refactorizada distribuye las responsabilidades entre clases específicas e incorpora las técnicas de refactorización requeridas. La comparación realizada confirma que ambas versiones mantienen los mismos resultados en las 17 operaciones evaluadas y en el reporte, sin diferencias en los mensajes ni en los datos comparados.

El alcance del proyecto es acotado: no contempla usuarios reales ni una base de datos, y los datos empleados en las pruebas son sintéticos. La interfaz se mantiene igual en ambas versiones para centrar la comparación en los cambios internos del código.

9.2 Conclusiones individuales

Integrante 1. Ariana Maldonado

El desarrollo de Academa permitió comprender la importancia de evaluar tanto el funcionamiento de un sistema como la calidad interna de su código. A partir del análisis de las dos versiones, se pudo comprobar que un programa puede cumplir con sus funciones y, al mismo tiempo, presentar problemas de organización que dificultan su mantenimiento. La comparación entre ambas implementaciones permitió reconocer la importancia de mejorar la estructura sin modificar el comportamiento esperado.

Integrante 2: Matias Manuel Carro

El análisis de la deuda técnica permitió identificar cómo los nombres poco descriptivos, la concentración de responsabilidades y la duplicación de código afectan la comprensión y el mantenimiento de un sistema. La aplicación de los principios SOLID y las prácticas de Clean Code ayudó a reconocer alternativas para organizar mejor las clases y facilitar futuras modificaciones. También permitió comprender que no todos los principios presentan necesariamente un incumplimiento en cada proyecto.

Integrante 3: Tomas Esteban Bohorquez Tello

La aplicación de las técnicas de refactorización y la ejecución de las pruebas comparativas permitieron comprobar la importancia de realizar cambios controlados. Separar responsabilidades y mejorar la estructura de los métodos facilitó la lectura del código. Además, la comparación de las 17 operaciones y del reporte permitió verificar que las modificaciones conservaron los resultados evaluados. La revisión de código complementó este proceso al ofrecer criterios concretos para evaluar las mejoras introducidas.

Integrante 4: Luciano Andrelo

La realización de las pruebas funcionales y comparativas de Academa permitió comprender la importancia de verificar el comportamiento del sistema durante un proceso de refactorización. A partir de seis pruebas sobre la versión inicial y una comparación de 17 operaciones entre ambas implementaciones, se pudo comprobar que los cambios realizados conservaron los resultados esperados en los casos evaluados.

Además, la aplicación del checklist de code review permitió identificar mejoras en la calidad interna del código, registrar observaciones y justificar las decisiones adoptadas. Esta experiencia destacó la importancia de las pruebas automatizadas, la revisión técnica y la documentación para mantener la calidad del software durante su evolución.

10. Repositorio, tags y video

El proyecto cuenta con un repositorio que reúne el código fuente de Academa, el historial de cambios, las versiones inicial y refactorizada, y la documentación técnica. La organización del repositorio permite seguir las distintas etapas del trabajo, desde el diagnóstico de los problemas de calidad hasta la refactorización y la verificación del comportamiento.

Los siguientes *tags* identifican los principales hitos del proyecto:

Tag	Versión o etapa identificada
estado-inicial	Versión inicial, previa a la refactorización.
estadorefactorizado	Versión final refactorizada.
etapa-diagnostico	Diagnóstico de los code smells identificados.
etapa-refactoring	Aplicación de las cuatro técnicas de refactorización.
etapa-pruebas	Pruebas comparativas y verificación de la conservación del comportamiento.
etapa-docs	Documentación del proyecto, README e informe final.

Enlace al repositorio: AriMaldo19/Academa: Trabajo Práctico Integrador - Metodología de Sistemas II. Análisis de calidad, deuda técnica y refactorización del sistema Academa.

11. Uso de inteligencia artificial

Se utilizó ChatGPT como herramienta de apoyo para analizar la estructura del código, evaluar mejoras en la distribución de responsabilidades y revisar la aplicación de las técnicas de refactorización. Las propuestas se contrastaron con el código del proyecto y los resultados de las pruebas comparativas, que permitieron verificar la conservación del comportamiento entre ambas versiones.

Prompt 1. Análisis del código original

"Actuá como un profesional de ingeniería de software especializado en calidad interna y diseño orientado a objetos en Java.

A partir del código fuente original del sistema Academa, ya desarrollado, analizá la organización de las clases y los métodos, la distribución de responsabilidades y la ubicación de la lógica de negocio. Identificá posibles problemas de diseño y calidad interna presentes en el código, explicando sus evidencias y cómo podrían afectar la legibilidad, la cohesión y la mantenibilidad del sistema.

El objetivo es apoyar el diagnóstico técnico del código existente, comprender los problemas detectados y fundamentar las oportunidades de mejora, sin desarrollar nuevas funcionalidades ni reescribir el sistema."

Prompt 2. Revisión de las refactorizaciones realizadas

"Actuá como un profesional de ingeniería de software especializado en refactorización, principios SOLID y patrones de diseño en Java.

A partir del código original y de la versión refactorizada del sistema Academa, revisá los cambios realizados por el equipo para evaluar si las técnicas de refactorización aplicadas responden a los problemas de diseño previamente identificados. Explicá qué aspectos de la organización del código y de la distribución de responsabilidades mejora cada cambio, considerando la legibilidad, la cohesión y la mantenibilidad.

Basá el análisis en las diferencias observables entre ambas versiones. No propongas una implementación completa alternativa ni des por realizadas modificaciones que no estén presentes en el código. El objetivo es complementar la revisión técnica de los cambios efectuados por el equipo."

12. Anexo de ejecución

Para ejecutar y verificar Academa se requiere **JDK 21** o una versión compatible que incluya **javac.** Las pruebas documentadas se realizaron con **OpenJDK 21.0.12.** La interfaz se abre en el navegador.

- **ejecutar-inicial.bat:** inicia la versión inicial de Academa en http://127.0.0.1:8081.
- **ejecutar-refactorizada.bat:** inicia la versión refactorizada en http://127.0.0.1:8082.
- **compilar-y-probar.bat:** compila el código y ejecuta la comparación entre ambas versiones.

Al operar cada versión, se generan los archivos solicitudes-inicial.txt y solicitudes-refactorizada.txt, respectivamente. La comparación automática utiliza archivos temporales independientes, por lo que no depende de esos archivos.

Cada línea de los archivos de persistencia contiene los siguientes campos, separados por el carácter |: identificador, nombre, legajo, tipo, prioridad, descripción, estado, responsable, etiqueta y marca. El reporte presenta la información en un formato legible. Por este motivo, la descripción no debe contener el carácter |, ya que podría alterar la separación de los campos.